

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE TEORIA

INFORMACIÓN BÁSICA					
ASIGNATURA:	FISICA COMPUTACIONAL				
TÍTULO DE LA PRÁCTICA:	Resolución de la Práctica 2: Laboratorio B				
NÚMERO DE LA PRÁCTICA:	02	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN:	05/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Jara Mamani Mariel Alisson				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Prof. Edwin Agapito Llamoca Requena					

RESULTADOS Y PRUEBAS
SOLUCIÓN DE LOS EJERCICIOS PROPUESTOS 0.1 Ejercicio 1: Circunferencia de radio 3 Problema: Graficar una circunferencia con radio $r = 3$ cuyo centro esta en el origen. Implementación: Se definieron las funciones de aceleración a_x y a_y que calculan la aceleración gravitatoria. Se utilizó el método de Euler para integrar las ecuaciones de movimiento con un paso temporal $h = 0.01$. Las trayectorias se inicializaron desde el punto (0, -3) con diferentes velocidades iniciales en el eje x (v_{x0} variando de 0.5 a 1.5). Debido al comportamiento de la condiciones iniciales, se dibujaron múltiples trayectorias alrededor de una circunferencia de radio $r = 3$ con centro en el origen. Código fuente: exe1.py <pre>import matplotlib.pyplot as plt import numpy as np plt.figure(figsize=(12, 10)) # Configuración inicial h = 0.01 k = 0.05 tfin = 30 # Datos de la circunferencia r = 3 centro_x, centro_y = 0, 0 # Dibujar el círculo circulo = plt.Circle((centro_x, centro_y), r, color='black', fill=False, linestyle='-', linewidth=3) plt.gca().add_patch(circulo) # Funciones de aceleración def ax(x, y): return -x / ((x**2 + y**2)**(3/2)) def ay(x, y): return -y / ((x**2 + y**2)**(3/2)) for vx0 in np.arange(0.5, 1.5, k): vx = vx0 vy = 0.0 x, y = 0.0, -3.0</pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

px = [x]
py = [y]

for t in np.arange(0, tfin, h):

    vx = vx + ax(x, y) * h
    vy = vy + ay(x, y) * h
    x = x + vx * h
    y = y + vy * h

    px.append(x)
    py.append(y)

# Graficar cada trayectoria
plt.plot(px, py, markersize=5)

plt.title("Ejercicio 1 - Circunferencia y trayectorias")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.grid(True)
plt.gca().set_aspect("equal", adjustable="box")

plt.savefig("TAREA-02/img/pr_b/exe1.png", dpi=300, bbox_inches="tight")
plt.show()

```

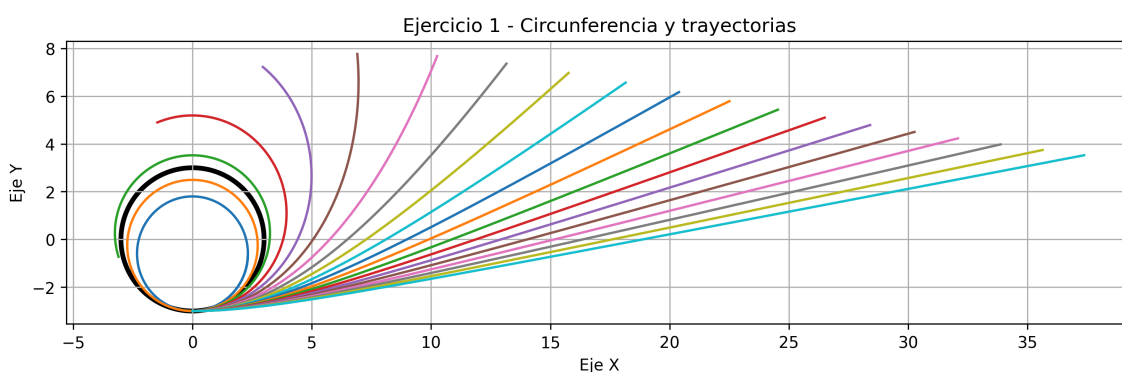


Figura 1: Gráfico de la circunferencia con radio $r = 3$ m

Resultados: En el gráfico se observa la circunferencia de radio 3 con centro en el origen (0,0). Las trayectorias de partículas siguen órbitas que pasan cerca de la circunferencia, algunas completan revoluciones y otras escapan. El patrón muestra cómo diferentes velocidades iniciales producen diferentes tipos de trayectorias alrededor del centro gravitatorio.

0.2 Ejercicio 2: Omisión grafica en $r \leq 3$

Problema: Cuando la trayectoria llegue a $r = \sqrt{(x-1)^2 + y^2} \leq 3$ que no grafique pero debe continuar la simulación.

Implementación: Se modificó la función $ax(x,y)$ para que calcule la aceleración considerando el centro en (1,0) según la fórmula $-(x-1)/r^3$. Se agregó la condición de omisión gráfica dentro del bucle: cuando la distancia al centro sea menor o igual a $r(\sqrt{(x-1)^2 + y^2}) \leq 3$, se deja de agregar puntos a la trayectoria (usando break) pero la simulación continúa para las demás partículas. Esto permite que las partículas que chocan no se grafiquen dentro del círculo.

Código fuente: exe2.py

```

import matplotlib.pyplot as plt
import numpy as np

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```
plt.figure(figsize=(12, 10))

# Configuración inicial
h = 0.01
k = 0.05
tfin = 200

# Datos de la circunferencia
r = 3
centro_x, centro_y = 0, 1

# Dibujar el círculo
circulo = plt.Circle((centro_x, centro_y), r, color='black', fill=False, linestyle='-', linewidth=1.5)
plt.gca().add_patch(circulo)

# Funciones de aceleración
def ax(x, y):
    return -(x-1) / ((x**2 + y**2)**(3/2))

def ay(x, y):
    return -y / ((x**2 + y**2)**(3/2))

for vx0 in np.arange(0.01, 1.5, k):
    vx = vx0
    vy = 0.0
    x, y = 0.0, -4.0

    px = [x]
    py = [y]

    for t in np.arange(0, tfin, h):
        vx = vx + ax(x, y) * h
        vy = vy + ay(x, y) * h
        x = x + vx * h
        y = y + vy * h

        if np.sqrt((x-centro_x)**2 + (y-centro_y)**2) <= r:
            px.append(x)
            py.append(y)
            break

        px.append(x)
        py.append(y)

        if abs(x) > 30 or abs(y) > 30:
            break

    # Graficar cada trayectoria
    plt.plot(px, py, linewidth=1)

plt.title("Ejercicio 2 - Simulación de Colisión con Obstáculo Circular")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.grid(True)
plt.gca().set_aspect("equal", adjustable="box")

plt.savefig("TAREA-02/img/pr_b/exe2.png", dpi=300, bbox_inches="tight")
plt.show()
```

Ejercicio 2 - Simulación de Colisión con Obstáculo Circular

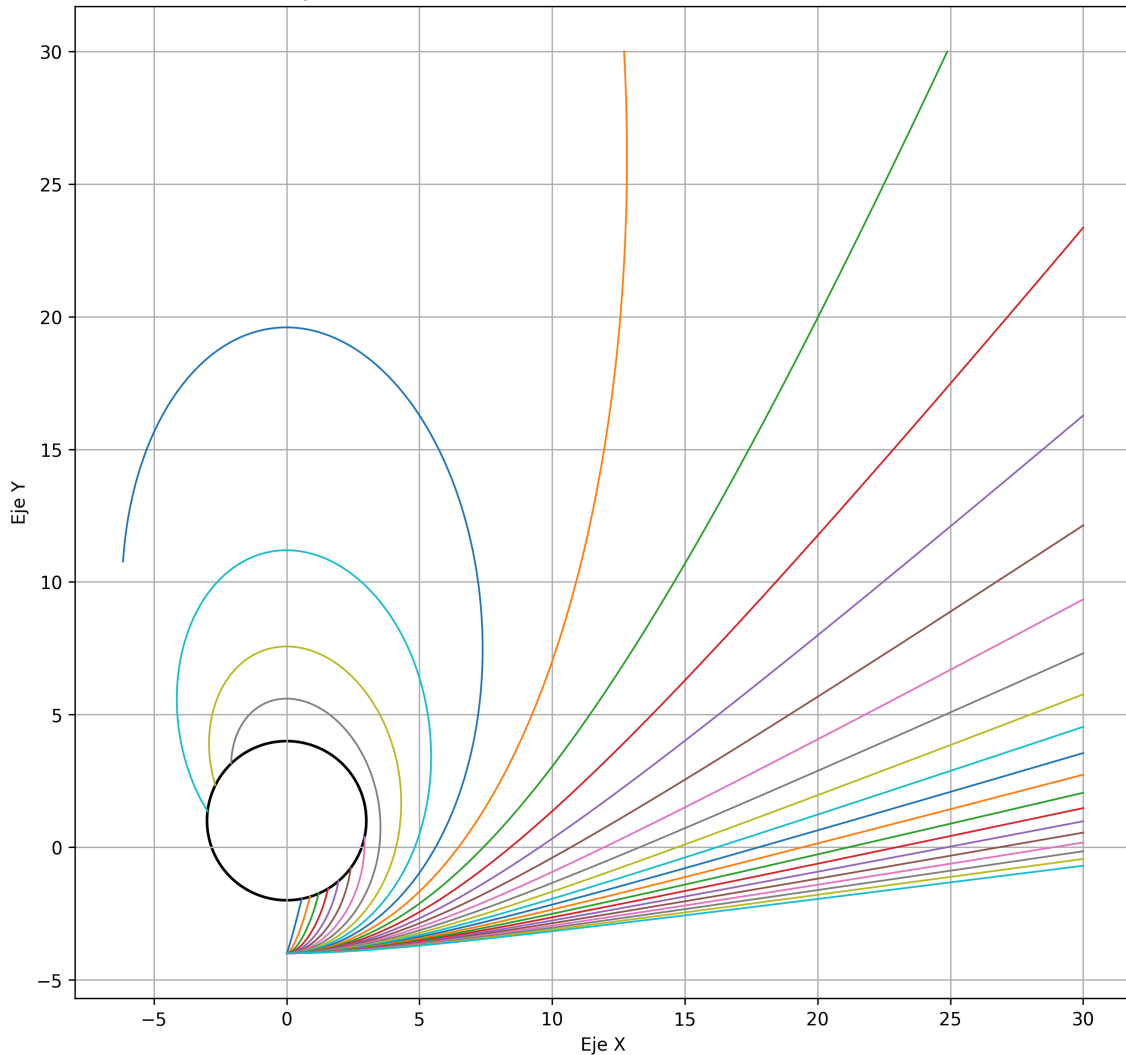


Figura 2: Gráfico de la trayectoria omitiendo $r \leq 3 \text{ m}$

Descripción: El gráfico muestra múltiples trayectorias evitando el interior del círculo de radio 3 con centro en (1,0). Como se puede observar, las líneas no penetran en el área sombreada del círculo, lo que indica que la condición de omisión funciona correctamente. Las partículas que chocan contra el obstáculo se detienen en el borde sin ser graficadas en el interior.

0.3 Ejercicio 3: Parábolas, Elipses e Hiperbolas

Problema: Grafique en un misma ventana, dos parábolas, dos elipses y dos hiperbolas

Implementación: Se-modificó la función $ax(x,y)$ para que calculara la aceleración considerando el centro en (0,1) en lugar de (0,0). Se implementó la clasificación de trayectorias basándose en el comportamiento: las parábolas tienen velocidad baja y chocan con el obstáculo circular ($r \leq 3$), las elipses tienen velocidad media y permanecen en órbita cerca del centro (distancia final ≤ 20), y las hiperbolas tienen velocidad alta y escapan del sistema. Se almacenaron las primeras dos trayectorias de cada tipo para graficarlas.

Código fuente: exe3.py

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```

import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(12, 8))

h = 0.01
k = 0.05
tfin = 450
r = 3
centro_x, centro_y = 0, 1

circulo = plt.Circle((centro_x, centro_y), r, color='black', fill=False, linestyle='--', linewidth=1.5, label='Circulo (r=3)')
plt.gca().add_patch(circulo)

def ax(x, y):
    return -(x) / ((x**2 + y**2)**(3/2))

def ay(x, y):
    return -y / ((x**2 + y**2)**(3/2))

# Listas para almacenar las trayectorias clasificadas
parabolas = [] # Velocidad Baja (Choque)
elipses = [] # Velocidad Media (Órbita)
hiperbolas = [] # Velocidad Alta (Escape)

for vx0 in np.arange(0.01, 1.5, k):
    vx, vy = vx0, 0.0
    x, y = 0.0, -4.0

    px, py = [x], [y]
    colisiono = False

    for t in np.arange(0, tfin, h):
        vx = vx + ax(x, y) * h
        vy = vy + ay(x, y) * h
        x = x + vx * h
        y = y + vy * h

        dist_obj = np.sqrt((x - centro_x)**2 + (y - centro_y)**2)

        # Condición de colisión (Parábolas de baja velocidad)
        if dist_obj <= r:
            px.append(x)
            py.append(y)
            colisiono = True
            break

        px.append(x)
        py.append(y)

        if abs(x) > 30 or abs(y) > 30: # Límite de escape
            break

    # Clasificar
    dist_final = np.sqrt(x**2 + y**2)

    if colisiono:
        if len(parabolas) < 2:
            parabolas.append((px, py))
    elif dist_final <= 20: # Se mantiene cerca (Elipse)
        if len(elipses) < 2:
            elipses.append((px, py))
    else: # Escapa (Hiperbola)
        if len(hiperbolas) < 2:
            hiperbolas.append((px, py))

# --- GRAFICAR RESULTADOS ---
for p in parabolas:
    plt.plot(p[0], p[1], color='orange', label='Parábola (Baja/Choque)')
for e in elipses:
    plt.plot(e[0], e[1], color='purple', label='Elipse (Media/Órbita)')

```

```
for h_curv in hiperbolas:
    plt.plot(h_curv[0], h_curv[1], color='skyblue', label='Hipérbola (Alta/Escape)')

# Evitar duplicados en la leyenda
handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
plt.legend(by_label.values(), by_label.keys())

plt.title("Ejercicio-3: 2 Parábolas, 2 Elipses, 2 Hipérbolas")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.grid(True)
plt.gca().set_aspect("equal", adjustable="box")
plt.savefig("TAREA-02/img/pr_b/exe3.png", dpi=300, bbox_inches="tight")

plt.show()
```

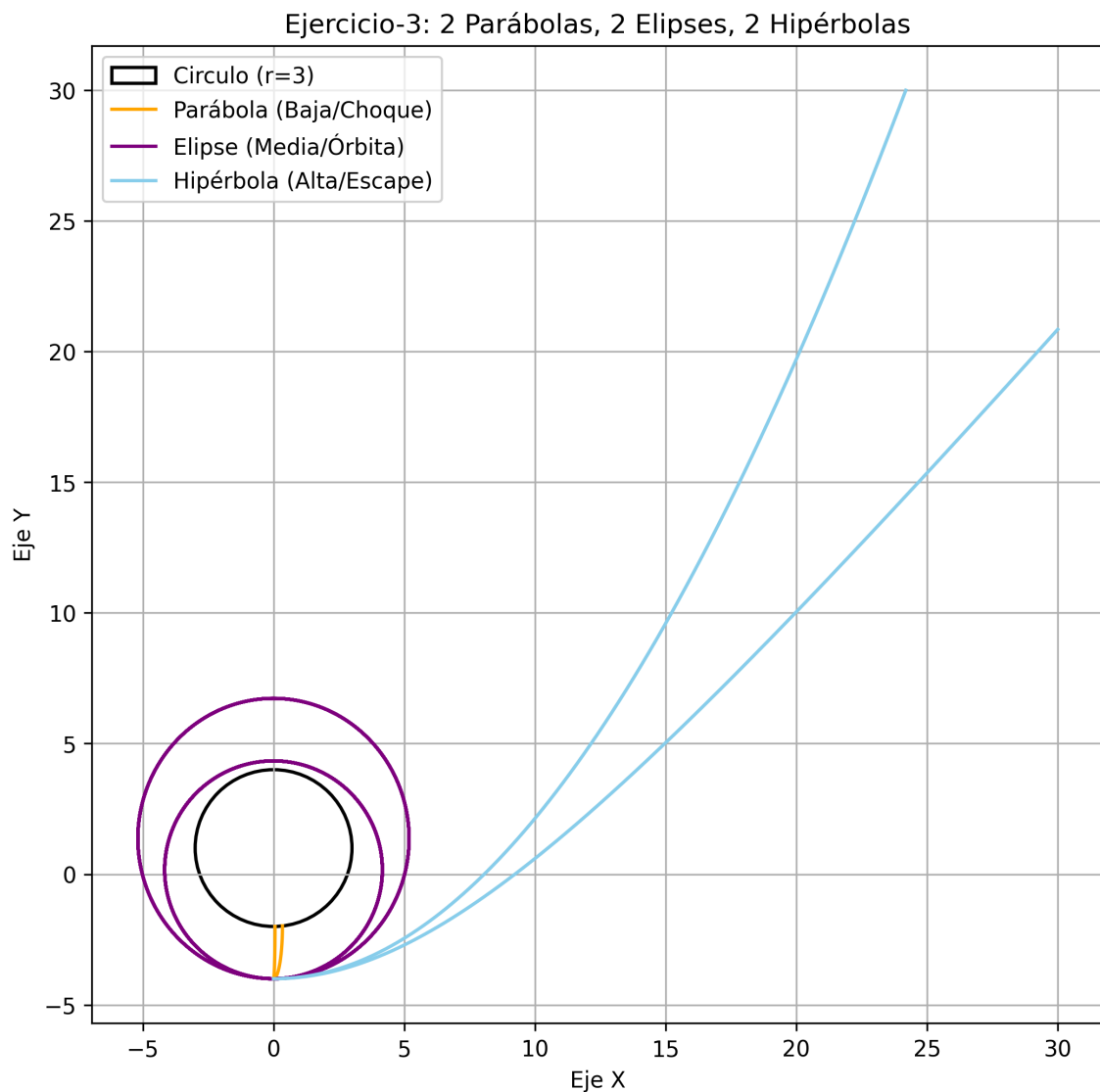


Figura 3: Dos parábolas, dos elipses y dos hipérbolas en una misma ventana

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

Descripción: El gráfico muestra dos parábolas de color naranja que representan las trayectorias de baja velocidad que chocan con el obstáculo circular. Se observa también dos elipses de color morado que corresponden a las órbitas estables donde la partícula gira alrededor del centro sin chocar ni escapar. Las dos hipérbolas de color azul claro representan las trayectorias de alta velocidad que escapan del campo gravitatorio. Como se puede observar, las parábolas son las más cortas porque terminan en colisión, las elipses son cerradas y recurrentes, y las hipérbolas se abren y se alejan del centro.

0.4 Ejercicio 4: Repetición con $v_x = v_y$

Problema: Repita el paso anterior para $v_x = v_y$. Debe notarse claramente el efecto de v_x

Implementación: Se-agregó la condición para que la velocidad inicial en y sea igual a la velocidad inicial en x ($v_y = v_{x0}$), de modo que las partículas partan con velocidad diagonal. Se-modificaron los puntos de guardado usando variables individuales (parabola_choque_1_x, parabola_choque_1_y, etc.) en lugar de listas para almacenar las dos parábolas, dos elipses y dos hipérbolas. El límite de escape se aumentó a 70 para permitir que las hipérbolas se observen mejor.

Código fuente: exe4.py

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 10))

# Configuración inicial
h = 0.01
k = 0.05
tfin = 1000
r = 3
centro_x, centro_y = 0, 1

circulo = plt.Circle((centro_x, centro_y), r, color='black', fill=False, linestyle='-', linewidth=1.5, label='Circulo (r=3)')
plt.gca().add_patch(circulo)

def ax(x, y):
    return -(x) / ((x**2 + y**2)**(3/2))

def ay(x, y):
    return -y / ((x**2 + y**2)**(3/2))

parabola_choque_1_x, parabola_choque_1_y = [], []
parabola_choque_2_x, parabola_choque_2_y = [], []
elipse_orbita_1_x, elipse_orbita_1_y = [], []
elipse_orbita_2_x, elipse_orbita_2_y = [], []
hiperbola_escape_1_x, hiperbola_escape_1_y = [], []
hiperbola_escape_2_x, hiperbola_escape_2_y = [], []

for vx0 in np.arange(0.01, 1.5, k):
    # Cambio: vx = vy
    vx = vx0
    vy = vx0
    x, y = 0.0, -4.0

    px = [x]
    py = [y]
    colisiono = False

    for t in np.arange(0, tfin, h):
        vx = vx + ax(x, y) * h
        vy = vy + ay(x, y) * h
        x = x + vx * h
        y = y + vy * h

        dist_obj = np.sqrt((x - centro_x)**2 + (y - centro_y)**2)

        if dist_obj <= r:
            px.append(x)
            py.append(y)
            colisiono = True
            break
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```

px.append(x)
py.append(y)

if abs(x) > 60 or abs(y) > 60:
    break

radio_final = np.sqrt(x**2 + y**2)

if colisiono:
    if not parabola_choque_1_x:
        parabola_choque_1_x, parabola_choque_1_y = px, py
    elif not parabola_choque_2_x:
        parabola_choque_2_x, parabola_choque_2_y = px, py

elif radio_final <= 20:
    if not elipse_orbita_1_x:
        elipse_orbita_1_x, elipse_orbita_1_y = px, py
    elif not elipse_orbita_2_x:
        elipse_orbita_2_x, elipse_orbita_2_y = px, py

else:
    if not hiperbola_escape_1_x:
        hiperbola_escape_1_x, hiperbola_escape_1_y = px, py
    elif not hiperbola_escape_2_x:
        hiperbola_escape_2_x, hiperbola_escape_2_y = px, py

# ----- Gráficas -----
plt.plot(parabola_choque_1_x, parabola_choque_1_y, color='orange', label='Parábola (Choque)')
plt.plot(parabola_choque_2_x, parabola_choque_2_y, color='orange')
plt.plot(elipse_orbita_1_x, elipse_orbita_1_y, color='purple', label='Elipse (Órbita)')
plt.plot(elipse_orbita_2_x, elipse_orbita_2_y, color='purple')
plt.plot(hiperbola_escape_1_x, hiperbola_escape_1_y, color='skyblue', label='Hipérbola (Escape)')
plt.plot(hiperbola_escape_2_x, hiperbola_escape_2_y, color='skyblue')

plt.title("Ejercicio-3: Efecto vx = vy (2 de cada tipo)")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.grid(True)
plt.gca().set_aspect("equal", adjustable="box")

# Evitar duplicados en la leyenda
handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
plt.legend(by_label.values(), by_label.keys())
plt.savefig("TAREA-02/img/pr_b/exe4.png", dpi=300, bbox_inches="tight")
plt.show()

```

Ejercicio-3: Efecto $v_x = v_y$ (2 de cada tipo)

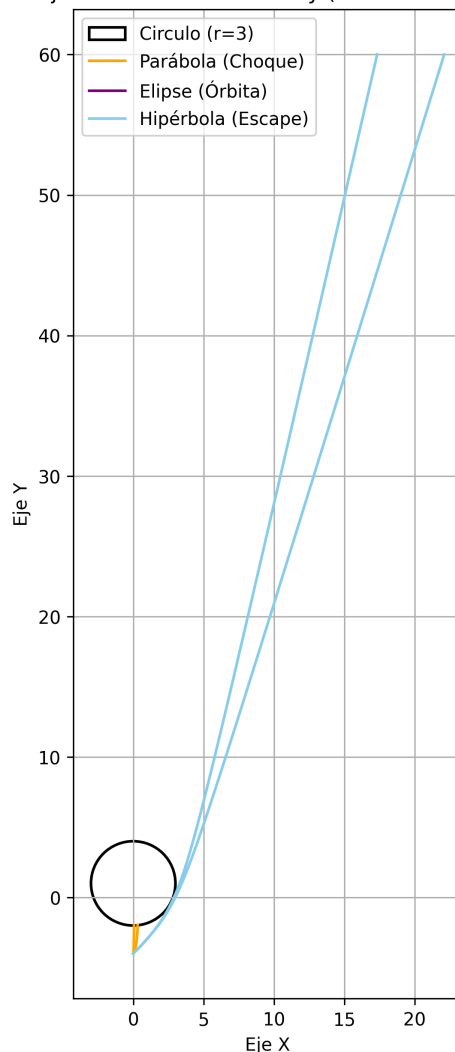


Figura 4: Dos parábolas, dos elipses y dos hipérbolas en una misma ventana con $v_x = v_y$

Descripción: El gráfico se observa cómo al hacer $v_y = v_x$, las trayectorias ahora tienen un componente diagonal desde el inicio. Las parábolas de color naranja siguen chocando con el obstáculo pero con un ángulo diferente. Las elipses de color morado muestran órbitas más simétricas debido a la velocidad inicial diagonal. Las hipérbolas de color azul claro escapan del sistema con trayectorias más amplias. Como se ve, el efecto de $v_x = v_y$ produce trayectorias más simétricas y balanceadas en comparación con el ejercicio anterior donde solo se tenía v_x .

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

- Se demostró que la simulación gravitatoria puede producir diferentes tipos de trayectorias (parábolas, elipses, hipérbolas) dependiendo de la velocidad inicial de la partícula. Las partículas con baja velocidad chocan (parábolas), las de velocidad media orbitan (elipses), y las de alta velocidad escapan (hipérbolas).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

2. El método de Euler funciona correctamente para integrar las ecuaciones de movimiento en problemas gravitatorios, permitiendo visualizar las órbitas y sus transiciones. Sin embargo, para simulaciones de larga duración pueden acumularse errores numéricos que afectan la precisión.
3. El cambio de $v_y = v_x$ produce trayectorias más simétricas debido a que la partícula comienza con un componente diagonal, mostrando el efecto de la dirección inicial en la forma de la órbita.

RECOMENDACIONES

1. Para mayor precisión en simulaciones de larga duración, se recomienda usar métodos de integración más sofisticados como Runge-Kutta de cuarto orden (RK4) en lugar del método de Euler.
2. Es importante ajustar adecuadamente los parámetros de tiempo (h) y el número máximo de iteraciones para evitar errores de overflow o tiempos de cómputo excesivos.
3. Se recomienda validar los resultados con valores teóricos conocidos (como el período de una órbita circular) para asegurar la correcta implementación del modelo físico.